

MANUAL TÉCNICO — GYMPOS

Sistema de Gestión de Gimnasio

Tecnología: Java 21 + JavaFX 21

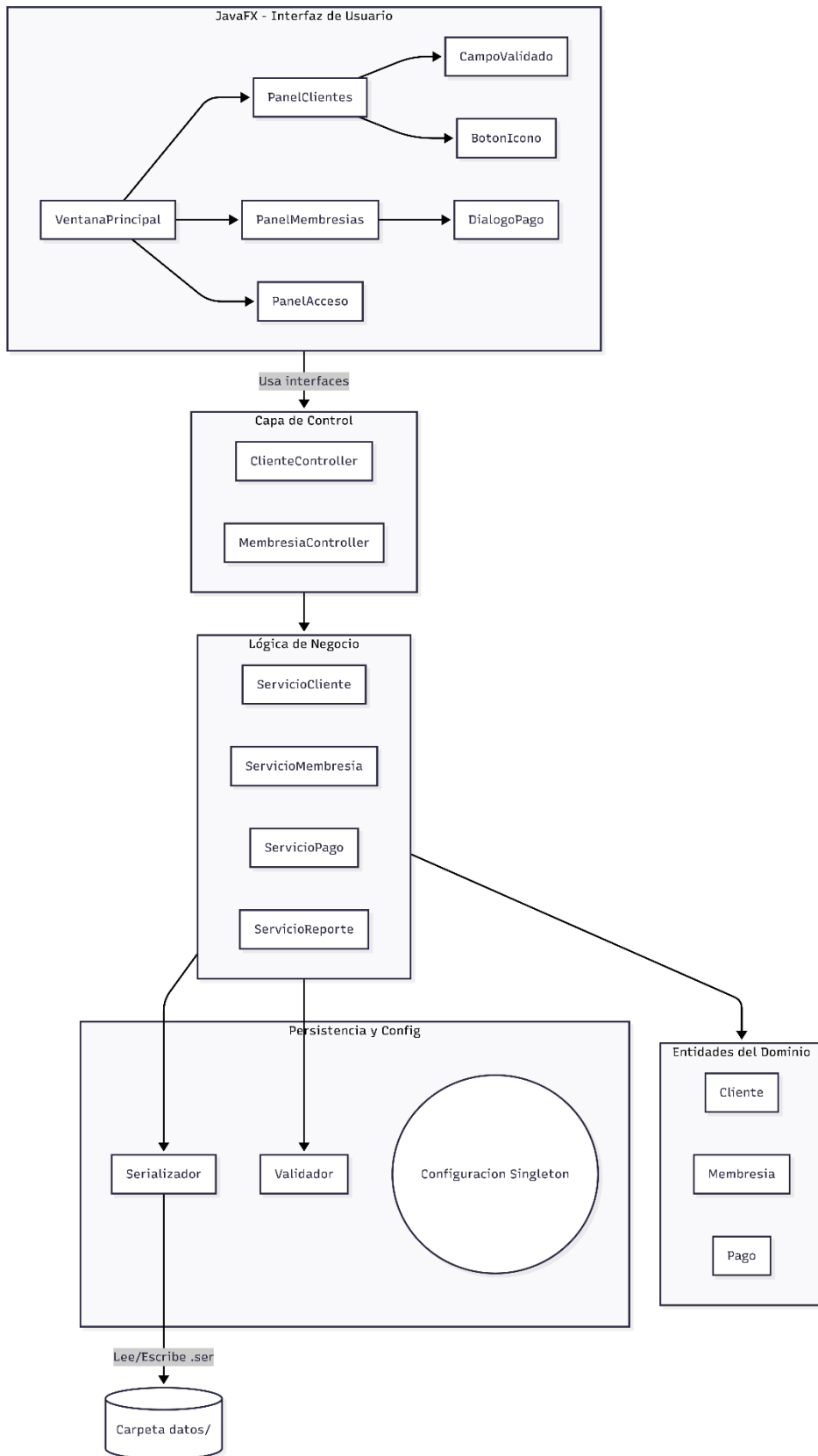
Nombre del gimnasio: IronPeak Gym

1. DESCRIPCIÓN GENERAL

GymPOS es una aplicación de escritorio desarrollada en Java 21 con interfaz gráfica JavaFX 21. El sistema permite la gestión completa de un gimnasio incluyendo clientes, membresías, pagos, control de acceso, inventario de equipos y calendario de clases grupales.

El sistema implementa el patrón de diseño MVC (Modelo-Vista-Controlador), persistencia mediante serialización Java, procesamiento asíncrono con `ExecutorService` y validación de formularios en tiempo real.

2. ARQUITECTURA DEL SISTEMA



3. ESTRUCTURA DE PACKAGES

gympos/

Main.java (Punto de entrada de la aplicación)

config/

Configuracion.java (Singleton de configuración del sistema)

modelo/

cliente/

Cliente.java (Entidad cliente)

membresia/

Membresia.java (Entidad membresía)

TipoMembresia.java (Enum con planes y precios)

pago/

Pago.java (Entidad pago con enums internos)

acceso/

RegistroAcceso.java (Entidad registro de entrada/salida)

equipo/

Equipo.java (Entidad equipo de gimnasio)

clase/

ClaseGrupal.java (Entidad clase grupal con inscritos)

servicio/

ServicioCliente.java (CRUD y validaciones de clientes)

ServicioMembresia.java (Gestión de membresías y renovaciones)

ServicioPago.java (Procesamiento asíncrono de pagos)

ServicioAcceso.java (Control de entrada y salida)

ServicioEquipo.java (Inventario de equipos)

ServicioClase.java (Calendario e inscripciones)

ServicioReporte.java (Generación asíncrona de reportes TXT)

controlador/

ClienteController.java (Controlador MVC para clientes)

MembresiaController.java (Controlador MVC para membresías)

excepcion/

GymPOSException.java (Excepción base del dominio)

ClienteNoEncontradoException.java (Excepción de búsqueda)

ValidacionException.java (Excepción de validación de campos)

PagoException.java (Excepción de procesamiento de pago)

util/

Serializador.java (Persistencia con ObjectOutputStream)

Validador.java (Reglas de validación reutilizables)

DatosIniciales.java (Generador de datos de prueba)

vista/

VentanaPrincipal.java (Ventana principal con menú)

cliente/

PanelClientes.java (Panel CRUD de clientes)

VistaCliente.java (Interfaz MVC de clientes)

membresia/

PanelMembresias.java (Panel CRUD de membresías)

VistaMembresia.java (Interfaz MVC de membresías)

acceso/

PanelAcceso.java (Panel control de acceso)

inventario/

PanelInventario.java (Panel inventario de equipos)

calendario/

PanelCalendario.java (Panel calendario de clases)

reporte/

PanelReportes.java (Panel generación de reportes)

dialogo/

DialogoPago.java (Diálogo modal de pago)

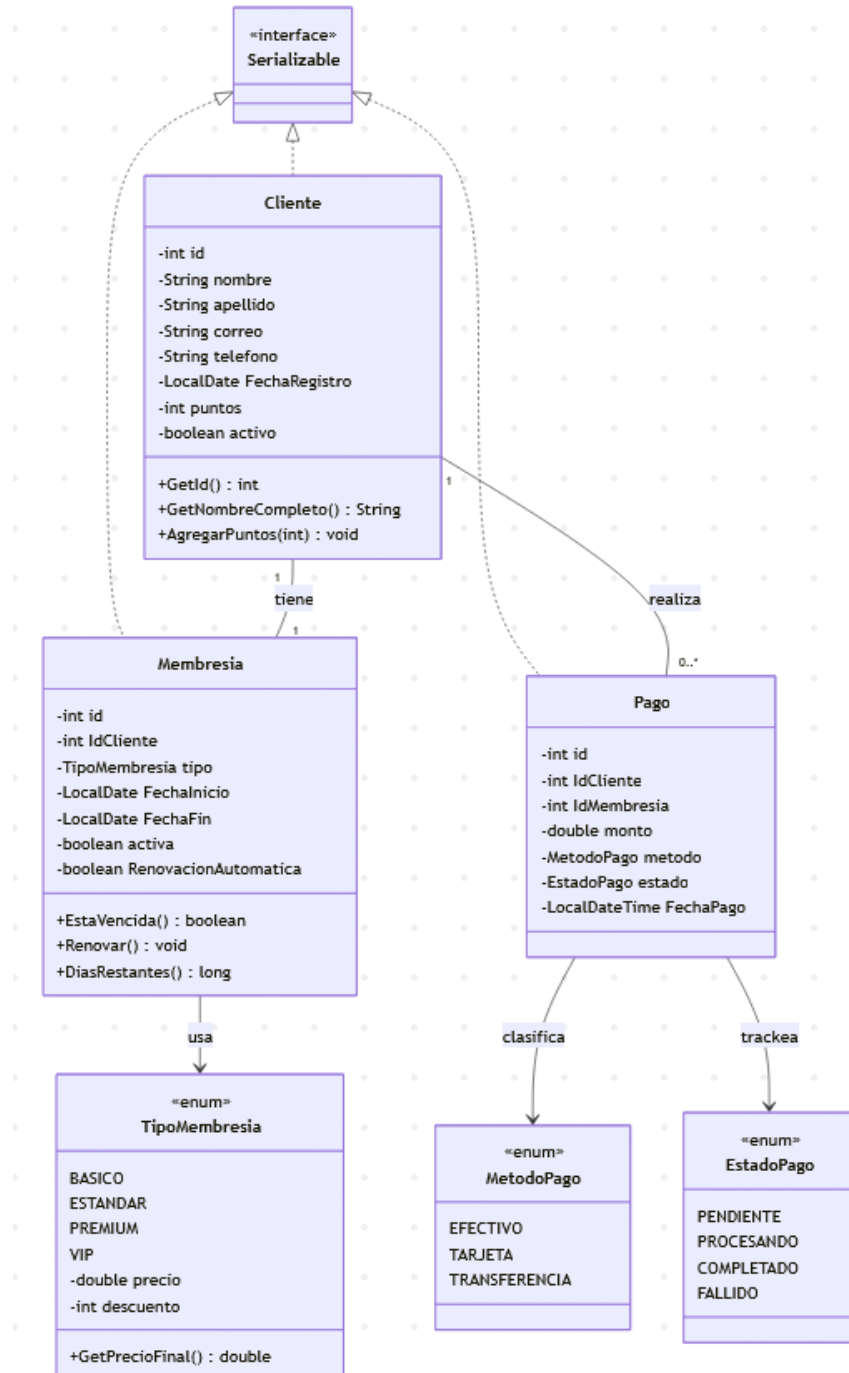
componente/

CampoValidado.java (TextField con validación en tiempo real)

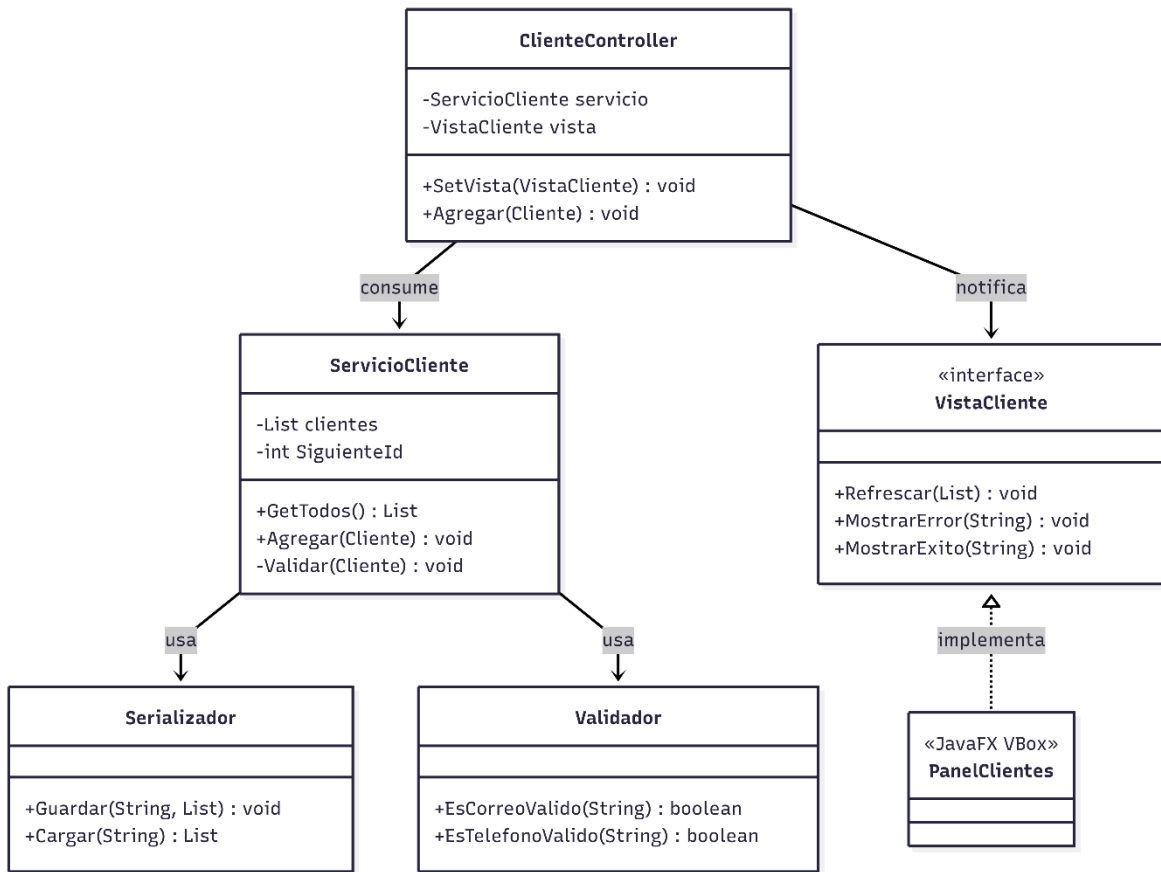
BotonIcono.java (Button con estilos según tipo)

Total: 37 clases en 12 packages.

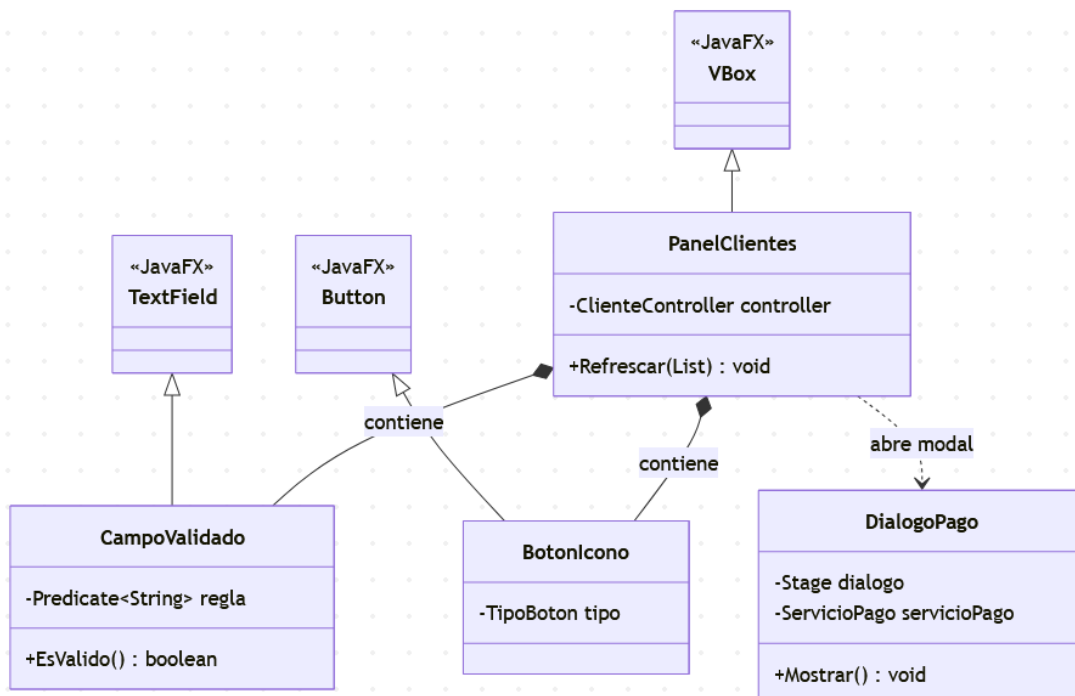
4. DIAGRAMA UML DE CLASES — MODELOS



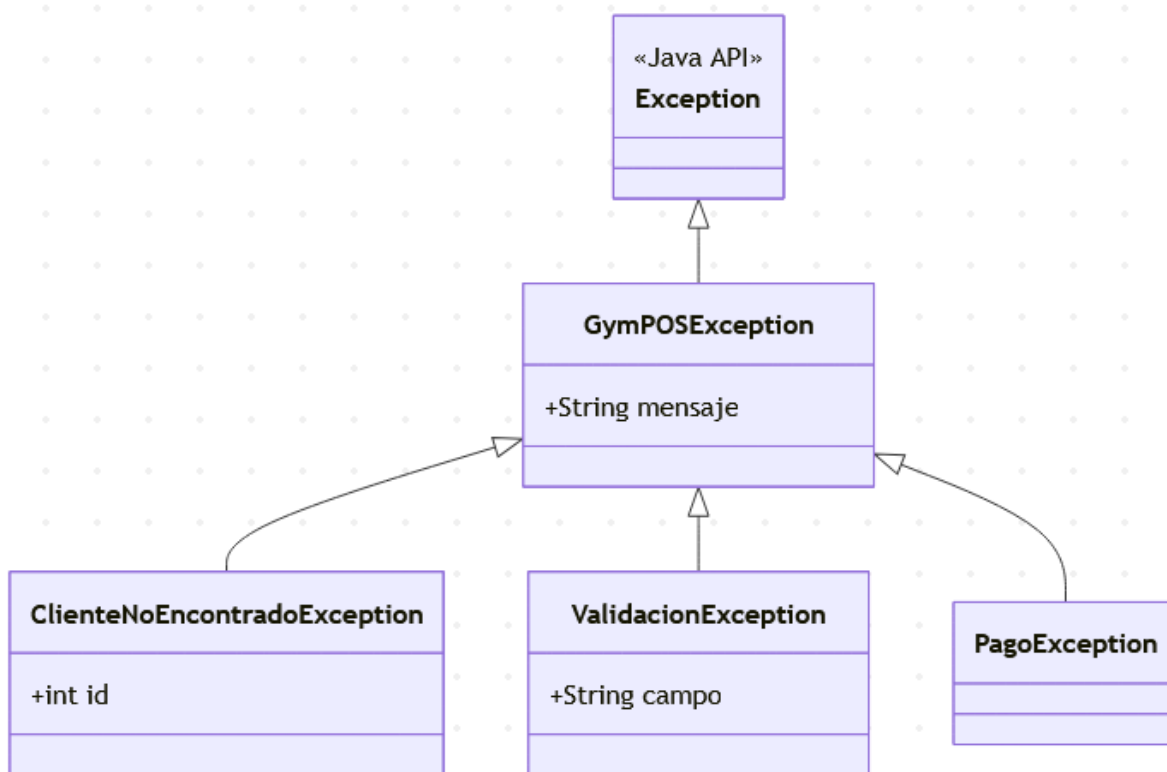
5. DIAGRAMA UML DE CLASES — SERVICIOS Y CONTROLADORES



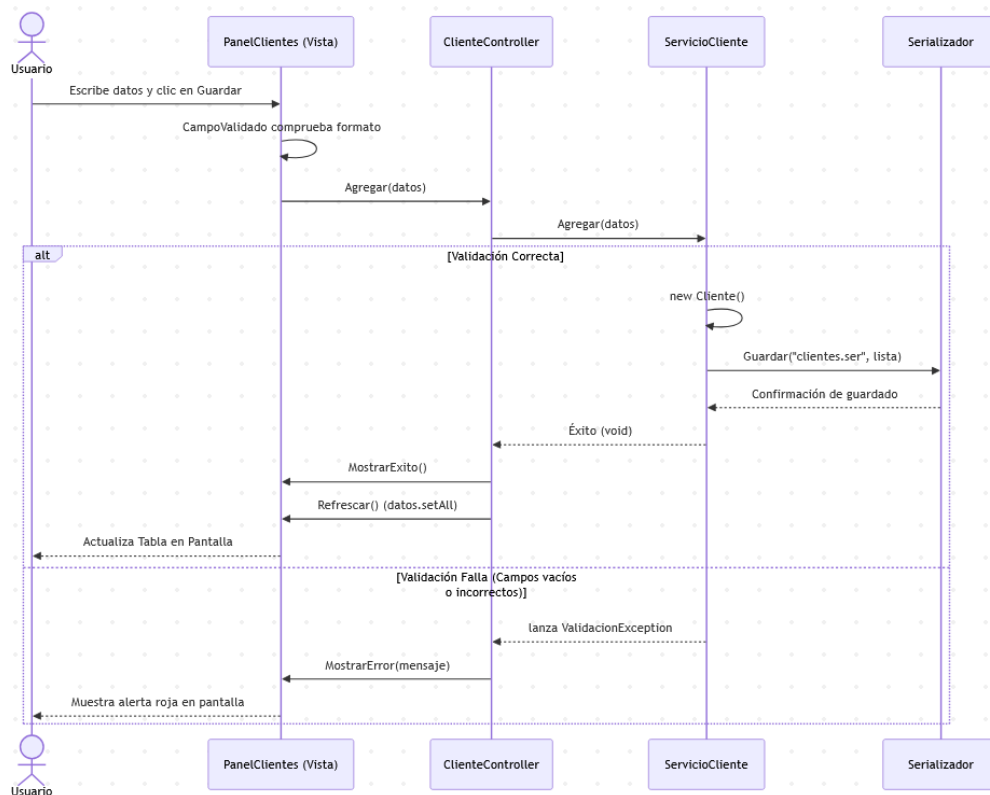
6. DIAGRAMA UML DE CLASES — VISTA



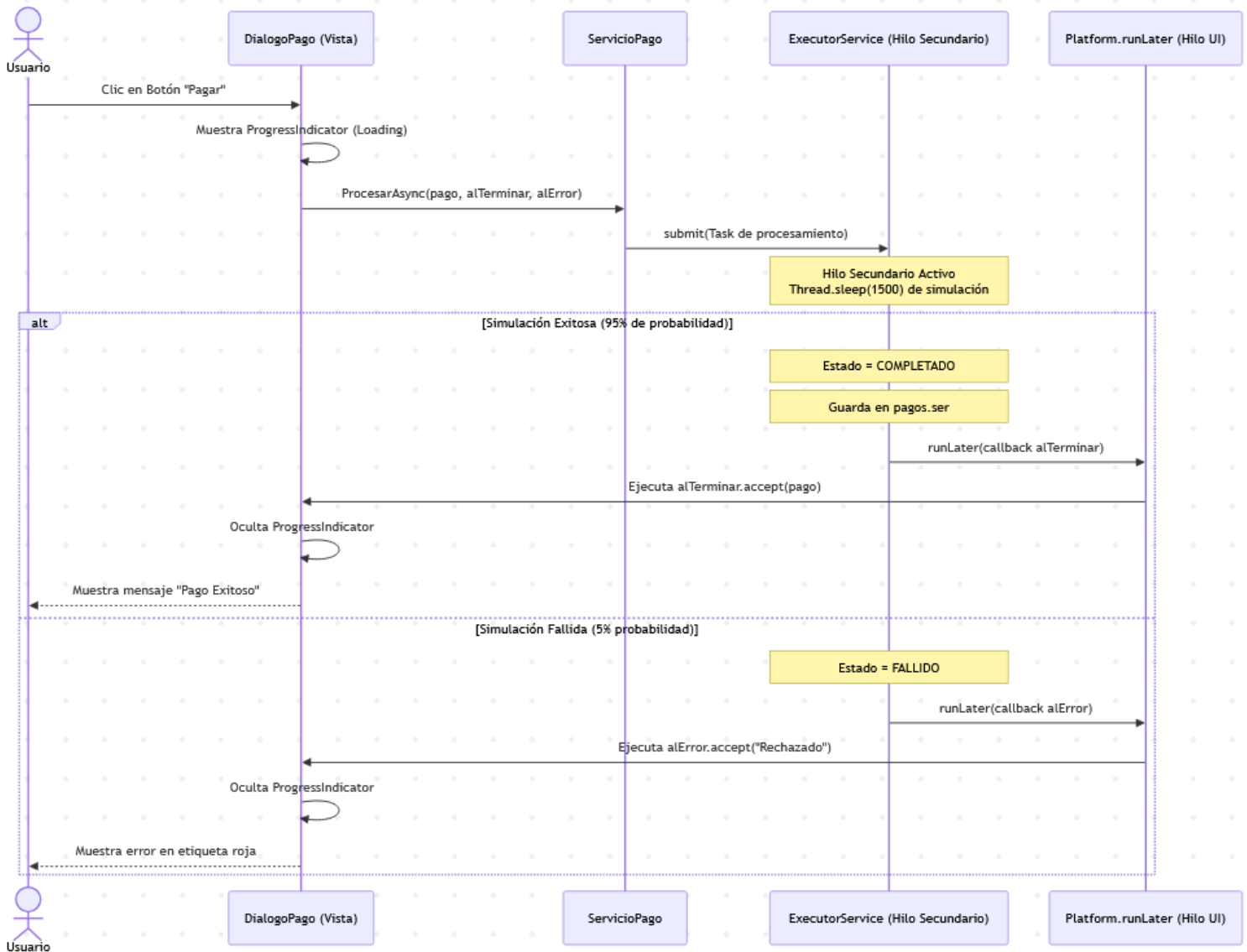
7. DIAGRAMA UML DE JERARQUÍA DE EXCEPCIONES



8. DIAGRAMA UML DE SECUENCIA — REGISTRO DE CLIENTE



9. DIAGRAMA UML DE SECUENCIA — PROCESAMIENTO DE PAGO



10. PATRÓN MVC IMPLEMENTADO

El patrón MVC se implementa de la siguiente manera:

- Modelo: Las clases en el package gympos.modelo son POJOs serializables que representan las entidades del dominio. No contienen lógica de interfaz ni de persistencia directa.
- Vista: Las clases en gympos.vista extienden nodos JavaFX (VBox, Stage, Button, TextField). Implementan interfaces como VistaCliente y VistaMembresia que definen los métodos que el controlador puede llamar para actualizar la pantalla: Refrescar(), MostrarError(), MostrarExito().

- Controlador: Las clases en gympos.controlador son el puente entre vista y servicio. Reciben eventos de la vista, llaman al servicio correspondiente, capturan las excepciones y notifican a la vista del resultado. El controlador nunca toca nodos JavaFX directamente.
- Servicio: Aunque técnicamente no es parte del MVC clásico, las clases en gympos.servicio encapsulan la lógica de negocio separándola del controlador. Esto mantiene los controladores delgados y la lógica centralizada y reutilizable.

Flujo de ejecución estándar:

Vista.Guardar() -> Controlador.Agregar(datos) -> Servicio.Agregar(datos) -> Serializador.Guardar()

11. DESCRIPCIÓN DETALLADA DE CLASES

- Main.java: Punto de entrada de la aplicación. Extiende javafx.application.Application. El método start() instancia VentanaPrincipal y llama a Mostrar(). El método main() llama a launch(args) que inicializa el toolkit de JavaFX.
- Configuracion.java: Implementa el patrón Singleton. Carga el archivo gympos.properties al iniciarse. Si el archivo no existe usa valores por defecto. Expone métodos Get(clave), Set(clave, valor) y GetInt(clave) para acceso tipado a la configuración.
- Cliente.java: Entidad principal del sistema. Recompensa puntos y concatena el nombre completo.
- Membresia.java: Entidad con lógica de dominio para evaluar vencimientos y gestionar renovaciones automáticas usando ChronoUnit.DAYS.
- TipoMembresia.java: Enum con cuatro planes (BÁSICO, ESTÁNDAR, PREMIUM, VIP) que calcula precios finales aplicando descuentos.
- Pago.java: Entidad con dos enums internos para rastrear el estado y el método del pago realizado.
- ServicioCliente.java: Implementa un CRUD completo auxiliándose de la clase Validador antes de guardar las colecciones en disco.
- ServicioPago.java: Utiliza subprocesos asíncronos mediante ExecutorService para procesar las transacciones y asegurar que la interfaz responda mediante Platform.runLater().
- ServicioReporte.java: Genera reportes en formato TXT en segundo plano de manera no bloqueante.
- ClienteController.java: Controlador delegado delgado encargado de capturar excepciones de la capa lógica y enviárselas a la vista.

- CampoValidado.java: Extiende TextField aplicando pseudo-clases CSS dinámicas si cumple el predicado de validación provisto.
- BotonIcono.java: Extiende Button asignando un estilo y texto predefinido según el enumerador TipoBoton.
- DialogoPago.java: Ventana emergente con propiedad de bloqueo modal que despliega un indicador de progreso de carga.

12. PERSISTENCIA DE DATOS

El sistema usa serialización Java nativa a través de la clase utilitaria Serializador. Si un archivo no existe al intentar cargarlo, se captura la IOException y retorna una lista vacía para que se autogeneren los datos de prueba iniciales.

13. MULTITHREADING

El sistema usa dos instancias de ExecutorService de un solo hilo de ejecución independiente para el procesamiento de pagos y reportes sin congelar las interacciones de los usuarios en la interfaz gráfica.

14. COMPONENTES JAVAFX PERSONALIZADOS

- CampoValidado: Extiende TextField agregando listeners reactivos a la propiedad del texto de entrada.
- BotonIcono: Extiende Button centralizando el catálogo de acciones del sistema bajo hojas de estilos (CSS) personalizadas.

15. FILTRADO INTERACTIVO

Implementado en todos los paneles con tabla usando la arquitectura encadenada: ObservableList -> FilteredList -> SortedList. Al invocar mutaciones sobre la lista original, los componentes visuales refrescan su estado automáticamente.

16. CONFIGURACIÓN DEL SISTEMA

El archivo gympos.properties se lee mediante instanciación perezosa (Lazy Singleton) garantizando consistencia centralizada de parámetros globales.

17. DATOS INICIALES DE PRUEBA

La clase DatosIniciales poblará el entorno con 20 clientes, 20 membresías, 20 pagos históricos, 8 equipos clasificados y 5 clases guiadas si el almacenamiento local está vacío.

18. REQUISITOS DEL SISTEMA

- JDK: Java 21 o superior.
- JavaFX SDK: Versión 21 compatible.
- Memoria RAM: Mínimo 256 MB.
- Disco Duro: 50 MB (App) + 80 MB (SDK).
- Dependencias Externas: Ninguna. Utiliza únicamente el JDK y el SDK nativo de JavaFX.